Sequence Diagrams

Four end-to-end flows of the whole application, each documented with its steps, data touched, ERD mapping and business rules

Contents

1. How to read the diagrams
2. Participants
3. Firestore paths used by the flows
4. Diagram 1 — Developer journey
5. Diagram 2 — Owner journey
6. Diagram 3 — Team workspace and tasks
7. Diagram 4 — Prompt evaluation and notifications
8. Observations and open points

1. How to read the diagrams

Each diagram is a UML sequence diagram. Time runs from top to bottom; each vertical dashed line is the lifeline of one participant. Horizontal bands (double lines) split the flow into numbered **phases**, and an **alt** box marks a step that runs only when its condition is true.

Symbol	Meaning
Solid arrow →	A request or call: a user action, a query, a write, or a function call.
Dashed arrow ⇢	A return value, a confirmation, or a push notification delivered to a person.
Arrow back to the same lifeline	A step the participant performs on itself (the Flutter App parsing a payload, for example).
Double-line band	Start of a numbered phase of the flow.
alt [condition]	Alternative fragment: the steps inside run only when the condition holds.
Note box	A comment attached to the participants it spans.

Conventions used in the step tables

- **Type** column: *Call* (solid arrow), *Return* (dashed reply), *Push* (dashed delivery to a person), *Self* (self message).
- Firestore paths such as *teams/{projectId}/tasks* are copied exactly as written on the diagrams. {braces} stand for a runtime id.
- The **ERD mapping** of each diagram lists the entities from DevSync_ERD.pdf that the flow reads or writes.
- Anything that is a proposal rather than something shown on a diagram is kept in section 8 (Observations).

2. Participants

Nine participants appear across the four diagrams. Persons are drawn as actors; systems are drawn as boxes.

Participant	Kind	Role in the system	Diagrams
Developer	Actor	Signs in, completes the profile, connects GitHub, discovers projects, requests to join, works in a team, rates AI output and reads notifications.	1, 4
Owner	Actor	Creates and publishes a project, finds missing roles, invites developers, and accepts join requests.	1, 2
Member	Actor	A team member who opens the team workspace and the Task Board and creates tasks.	3
Assignee	Actor	The member a task is assigned to; receives the push notification and moves the task between columns.	3
Flutter App	System	The Android client. Calls Firebase directly for reads and simple writes; calls Cloud Functions for anything that needs a secret or an LLM.	1–4
Firebase Auth	System	Email and Google sign-in; returns the session (uid).	1
Firestore DB	System	Main database: users, projects, teams, tasks, requests, invitations, notifications, feedback and prompt configuration.	1–4
Cloud Functions	System	Server-side logic: GitHub proxy, skill inference, AI matching and setup, FCM notifications, prompt-metric analysis.	1–4
LLM API / GitHub API	External	The language model (project setup, matching, pitch) and the GitHub REST data source. Drawn as one lifeline in Diagram 1 and as LLM API in Diagram 2.	1, 2

Overview of the four flows

#	Diagram	Main actor	Phases	Messages
1	Developer journey: authentication, GitHub, AI matching, join request	Developer	4	35
2	Owner journey: create, publish and staff a project	Owner	3	22
3	Team workspace, Task Board and task assignment	Member / Assignee	4	15
4	Prompt evaluation, prompt optimisation and notifications	Developer / system	3	12

3. Firestore paths used by the flows

Collection and document paths exactly as they appear on the diagrams, with the matching ERD entity from the companion file.

Firestore path	Used in	Written / read by	ERD entity
users/{uid}	1	App (check, save profile, generate publicId)	USER
users/{uid}/githubData	1	Cloud Function (save GitHub summary)	GITHUB_DATA, GITHUB_REPO
users/{uid}/detectedSkills	1	Cloud Function (status: pending)	DETECTED_SKILL, SKILL_EVIDENCE
requests/{projectId}_{uid}	1	App (save, then set Accepted)	JOIN_REQUEST
teams/{projectId}	2, 3	App (create team, fetch team info)	PROJECT_MEMBER, CONVERSATION
teams/{projectId}/members	1	App (add user after acceptance)	PROJECT_MEMBER
teams/{projectId}/tasks	3	App (stream and create tasks)	TASK
invitations/{projectId}_{inviteeId}	2	App (create invitation record)	INVITATION
notifications/{uid}/items	1, 2, 3, 4	Cloud Function writes; App streams unread items	NOTIFICATION
feedback/{id}	4	App (save rating with promptVersion)	AI_FEEDBACK
config/prompts	4	Cloud Function (update active prompt version)	PROMPT_VERSION (is_active)
projects (status: open, publishedAt)	2	App (save published project)	PROJECT

4. Diagram 1 — Developer journey

From first launch to joining a team: sign-in, profile, GitHub skill detection, AI project matching, and the request-to-join flow.

Participants

Developer · Flutter App · Firebase Auth · Firestore DB · Cloud Functions · LLM / GitHub API · Owner

Phases

Phase	Goal	What happens
1. Authentication & Profile	Get the user signed in with a complete profile.	Sign in with Google or email through Firebase Auth, then check *users/{uid}*. If the profile is incomplete (alt box), show the profile form and track selection, save the profile and generate the **publicId**.
2. GitHub Integration & AI Match	Detect skills from the user's GitHub activity.	The app calls **githubProxy**. The Cloud Function fetches repos, languages and activity, saves a summary in *githubData*, runs the skill-inference engine and stores the result in *detectedSkills* as **pending**. The developer reviews and accepts.
3. Project Discovery & AI Matching	Recommend open projects with explanations.	A Cloud Function compares the user profile with the open projects. It returns ranked projects, an AI explanation and the skill gap; the app displays them and the developer opens a project.
4. Collaboration Request & Workspace	Join a team.	The LLM drafts a collaboration pitch; the developer edits and submits. The request is stored, the owner is notified, accepts, and the developer is added to the team and enters the workspace.

ERD mapping

USER · USER_SKILL · GITHUB_DATA · GITHUB_REPO · DETECTED_SKILL · SKILL_EVIDENCE · AI_GENERATION · MATCH_RECOMMENDATION · MATCH_SKILL · JOIN_REQUEST · PROJECT_MEMBER · NOTIFICATION

Business rules shown on the diagram

- Sign-in is by **Google or email** only; GitHub is connected later, from inside the profile.
- A new user cannot continue until the profile form (including track) is submitted; a unique **publicId** is generated on save.
- Detected skills are saved as **pending**: the developer must review and accept them before they count as skills.
- Matching returns a **score with an explanation** and the **skill gap**, never a bare number.
- The pitch is AI-drafted but the developer edits and submits it; the request document id *{projectId}_{uid}* makes a second request for the same project impossible.
- Accepting a request updates its status, adds the user to *teams/{projectId}/members* and notifies the developer.

Diagram 1 — figure

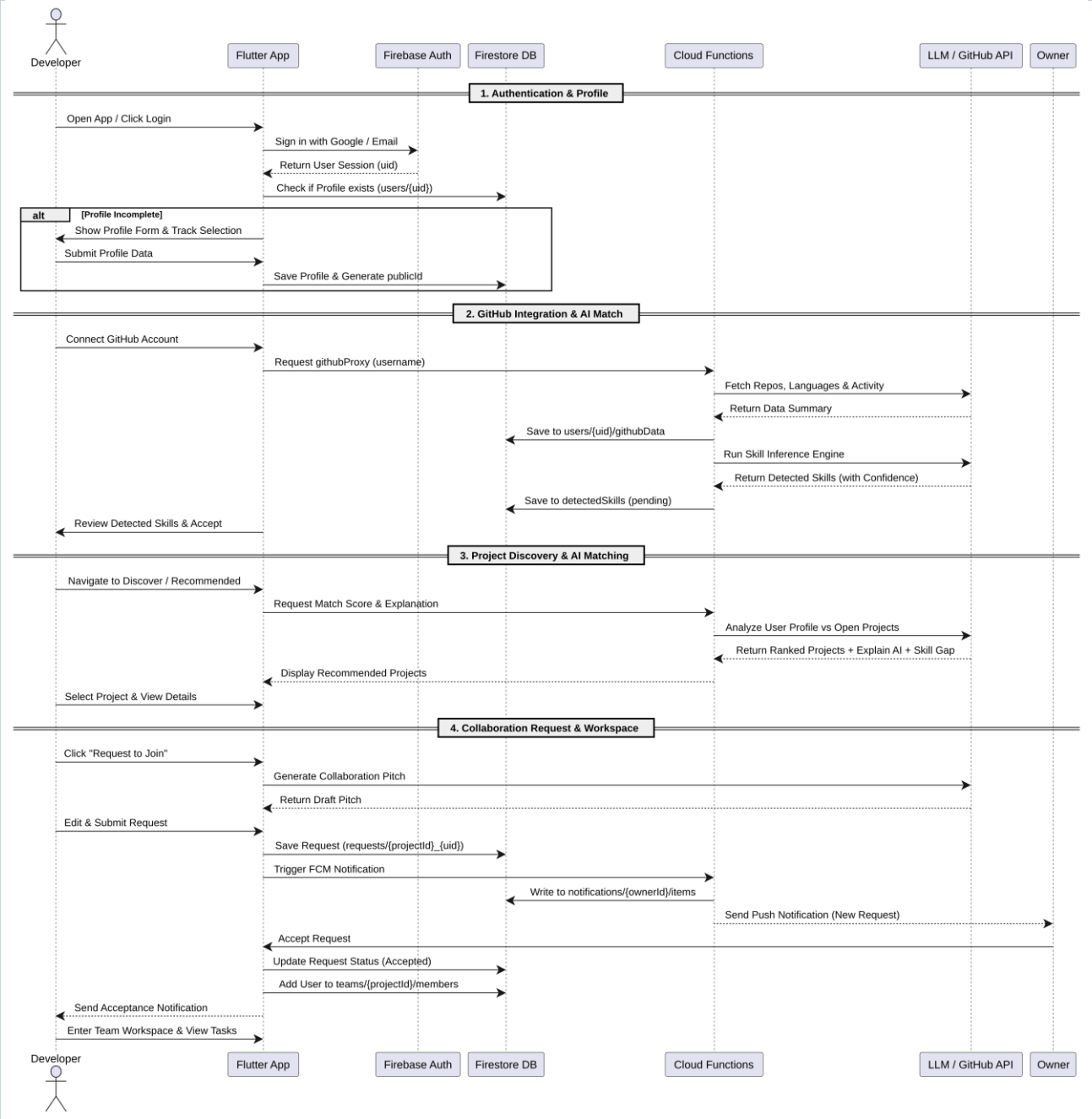


Figure 1. Developer journey: authentication, GitHub integration, AI matching and request to join.

Diagram 1 — message list

#	From	To	Message / action	Type
1. Authentication & Profile				
1	Developer	Flutter App	Open App / Click Login	Call
2	Flutter App	Firebase Auth	Sign in with Google / Email	Call
3	Firebase Auth	Flutter App	Return User Session (uid)	Return
4	Flutter App	Firestore DB	Check if Profile exists (users/{uid})	Call
alt [Profile Incomplete]				
5	Flutter App	Developer	Show Profile Form & Track Selection	Call
6	Developer	Flutter App	Submit Profile Data	Call
7	Flutter App	Firestore DB	Save Profile & Generate publicId	Call
2. GitHub Integration & AI Match				
8	Developer	Flutter App	Connect GitHub Account	Call
9	Flutter App	Cloud Functions	Request githubProxy (username)	Call
10	Cloud Functions	LLM / GitHub API	Fetch Repos, Languages & Activity	Call
11	LLM / GitHub API	Cloud Functions	Return Data Summary	Return
12	Cloud Functions	Firestore DB	Save to users/{uid}/githubData	Call
13	Cloud Functions	LLM / GitHub API	Run Skill Inference Engine	Call
14	LLM / GitHub API	Cloud Functions	Return Detected Skills (with Confidence)	Return
15	Cloud Functions	Firestore DB	Save to detectedSkills (pending)	Call
16	Flutter App	Developer	Review Detected Skills & Accept	Call
3. Project Discovery & AI Matching				
17	Developer	Flutter App	Navigate to Discover / Recommended	Call
18	Flutter App	Cloud Functions	Request Match Score & Explanation	Call
19	Cloud Functions	LLM / GitHub API	Analyze User Profile vs Open Projects	Call
20	LLM / GitHub API	Cloud Functions	Return Ranked Projects + Explain AI + Skill Gap	Return
21	Cloud Functions	Flutter App	Display Recommended Projects	Return
22	Developer	Flutter App	Select Project & View Details	Call
4. Collaboration Request & Workspace				
23	Developer	Flutter App	Click "Request to Join"	Call
24	Flutter App	LLM / GitHub API	Generate Collaboration Pitch	Call
25	LLM / GitHub API	Flutter App	Return Draft Pitch	Return
26	Developer	Flutter App	Edit & Submit Request	Call
27	Flutter App	Firestore DB	Save Request (requests/{projectId}_{uid})	Call
28	Flutter App	Cloud Functions	Trigger FCM Notification	Call
29	Cloud Functions	Firestore DB	Write to notifications/{ownerId}/items	Call
30	Cloud Functions	Owner	Send Push Notification (New Request)	Push
31	Owner	Flutter App	Accept Request	Call
32	Flutter App	Firestore DB	Update Request Status (Accepted)	Call
33	Flutter App	Firestore DB	Add User to teams/{projectId}/members	Call
34	Flutter App	Developer	Send Acceptance Notification	Push
35	Developer	Flutter App	Enter Team Workspace & View Tasks	Call

5. Diagram 2 — Owner journey

How a project owner creates a project manually or with AI, reviews and publishes it, and then finds developers for the roles that are still empty.

Participants

Owner · Flutter App · Firestore DB · Cloud Functions · LLM API

Phases

Phase	Goal	What happens
1. Create Project Entry	Start a project, optionally with AI.	The owner taps **Create Project** (Manual or AI). In the AI-Assisted alt branch the owner writes the idea as free text; the app calls **llmProjectSetup()**, the Cloud Function asks the LLM, and the app receives a generated title, description, tech stack, difficulty and roles.
2. Review & Publish Project	Publish an approved project.	The owner reviews and edits details and tracks, then taps **Publish**. The app saves the project (*status: open, publishedAt: now*) and creates the team document *teams/{projectId}* with the owner as a member.
3. Missing Roles & Developer Alert	Fill the empty roles.	The app checks **openTrackKeys (filled < count)**, shows the Missing Role alert, and on **Find Suitable Developers** queries users by track and skills. The owner invites a developer; an invitation record is created and a notification is written for the invitee.

ERD mapping

PROJECT · PROJECT_SKILL · PROJECT_ROLE · ROLE_SKILL · PROJECT_MEMBER · AI_GENERATION · USER · USER_SKILL · INVITATION · NOTIFICATION

Business rules shown on the diagram

- AI setup is **optional**: it appears only in the alt branch; the manual path skips straight to review.
- Everything the AI generates is shown for **review and editing**; nothing is published until the owner taps Publish.
- Publishing creates the **team document** with the owner already inside it, so chat and tasks work from day one.
- A role counts as missing while the number of filled slots is less than the number needed.
- Matching candidates are queried by **track and skills**; the invitation id is *{projectId}_{inviteeId}*.
- The invitee is notified through a Cloud Function (FCM) that writes to their notifications.

Diagram 2 — figure

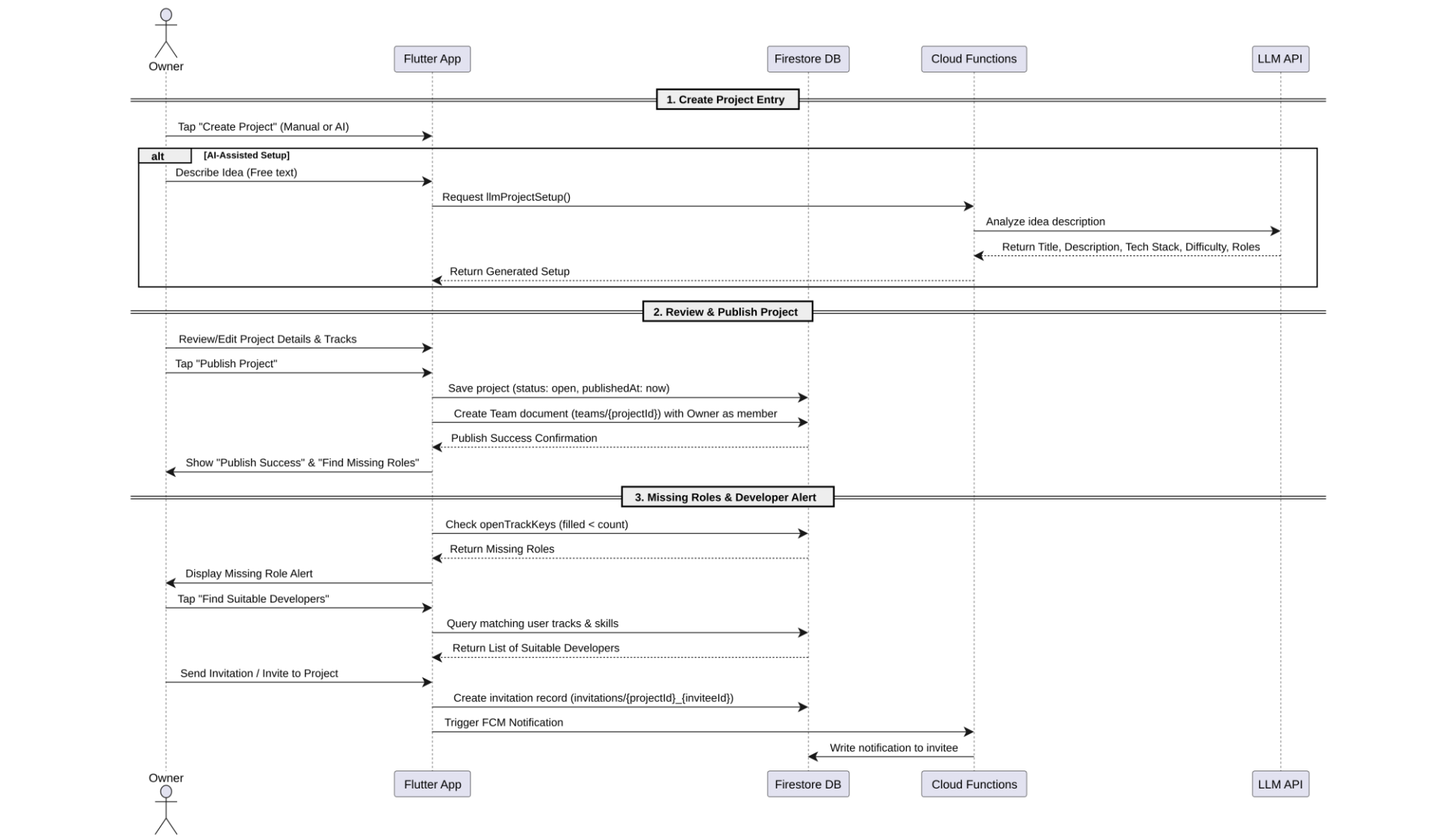


Figure 2. Owner journey: create, review, publish and find missing roles.

Diagram 2 — message list

#	From	To	Message / action	Type
1. Create Project Entry				
1	Owner	Flutter App	Tap "Create Project" (Manual or AI)	Call
alt [AI-Assisted Setup]				
2	Owner	Flutter App	Describe Idea (Free text)	Call
3	Flutter App	Cloud Functions	Request llmProjectSetup()	Call
4	Cloud Functions	LLM API	Analyze idea description	Call
5	LLM API	Cloud Functions	Return Title, Description, Tech Stack, Difficulty, Roles	Return
6	Cloud Functions	Flutter App	Return Generated Setup	Return
2. Review & Publish Project				
7	Owner	Flutter App	Review/Edit Project Details & Tracks	Call
8	Owner	Flutter App	Tap "Publish Project"	Call
9	Flutter App	Firestore DB	Save project (status: open, publishedAt: now)	Call
10	Flutter App	Firestore DB	Create Team document (teams/{projectId}) with Owner as member	Call
11	Firestore DB	Flutter App	Publish Success Confirmation	Return
12	Flutter App	Owner	Show "Publish Success" & "Find Missing Roles"	Call
3. Missing Roles & Developer Alert				
13	Flutter App	Firestore DB	Check openTrackKeys (filled < count)	Call
14	Firestore DB	Flutter App	Return Missing Roles	Return
15	Flutter App	Owner	Display Missing Role Alert	Call
16	Owner	Flutter App	Tap "Find Suitable Developers"	Call
17	Flutter App	Firestore DB	Query matching user tracks & skills	Call
18	Firestore DB	Flutter App	Return List of Suitable Developers	Return
19	Owner	Flutter App	Send Invitation / Invite to Project	Call
20	Flutter App	Firestore DB	Create invitation record (invitations/{projectId}_{inviteeId})	Call
21	Flutter App	Cloud Functions	Trigger FCM Notification	Call
22	Cloud Functions	Firestore DB	Write notification to invitee	Call

6. Diagram 3 — Team workspace and tasks

How a member enters a team workspace, watches the Task Board in real time, creates and assigns a task, and how the assignee is notified and updates the status.

Participants

Member · Flutter App · Firestore DB · Cloud Functions · Assignee

Phases

Phase	Goal	What happens
1. Enter Team Workspace	Open a team.	The member selects a project from **My Teams**. The app fetches *teams/{projectId}* (team info, members and active roles) and displays the workspace with Chat, Tasks and Members.
2. Task Management (Task Board)	See the board.	On opening the Task Board the app **streams** *teams/{projectId}/tasks*; tasks arrive grouped as To Do, In Progress and Done.
3. Create & Assign Task	Create a task and notify the assignee.	The member fills Title, Priority, Deadline and Assignee. The app saves the task with *status: todo*, triggers an FCM notification, and a Cloud Function writes *notifications/{assigneeId}/items* and sends the push.
4. Update Task Status	Move a task along the board.	The assignee drags or updates the status (for example In Progress → Done). The app updates Firestore and the change is reflected on the Task Board in real time.

ERD mapping

PROJECT_MEMBER · PROJECT_ROLE · TASK · NOTIFICATION · USER · CONVERSATION (opened from the workspace)

Business rules shown on the diagram

- The workspace shows **Chat, Tasks and Members** for the selected project.
- A new task always starts in **todo**; it has a title, priority, deadline and an assignee.
- Assigning a task triggers a **push notification** to the assignee, written through a Cloud Function.
- The Task Board is a **live stream**, so any status change appears for every member without a refresh.
- Both the member who creates a task and the assignee who updates it use the same Task Board.

Diagram 3 — figure

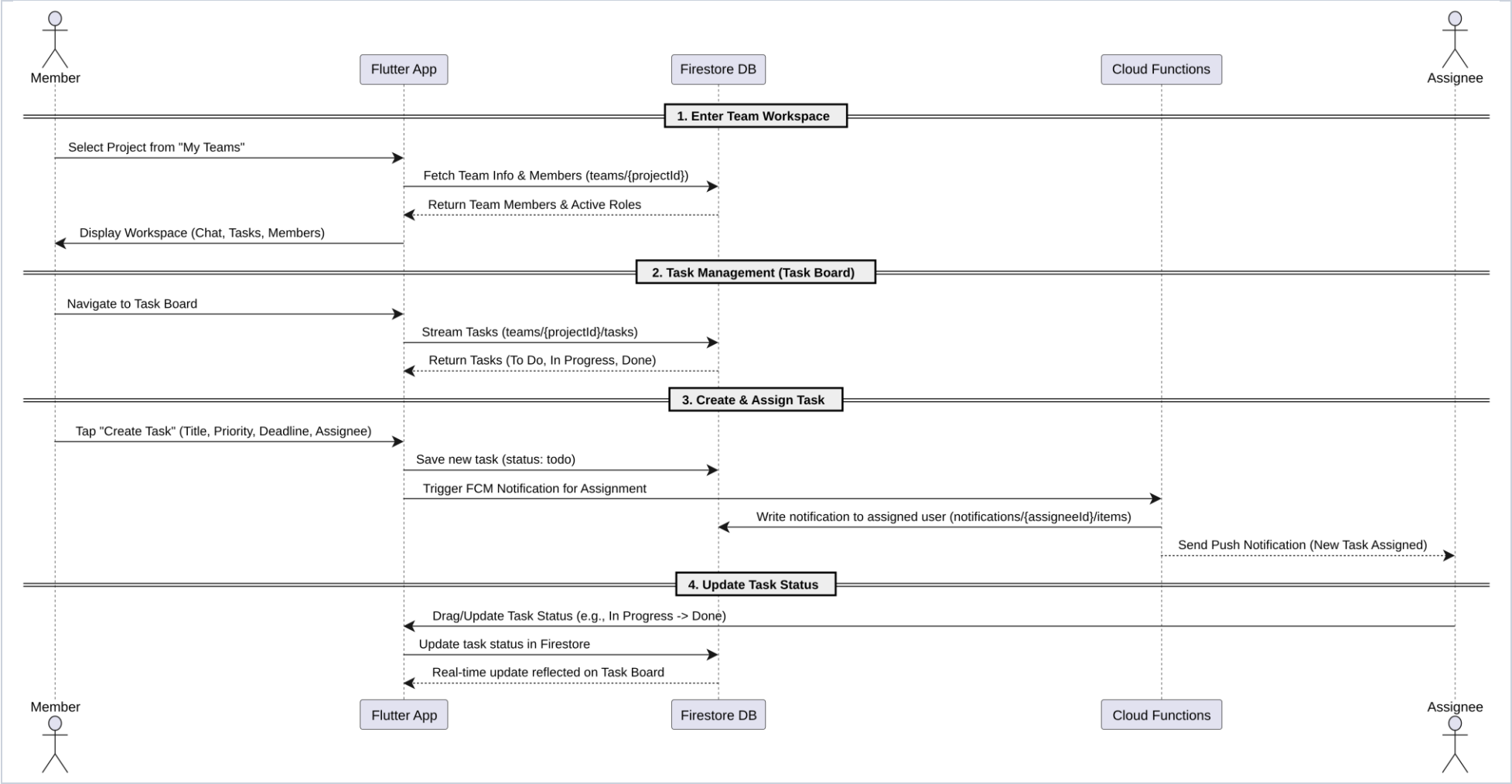


Figure 3. Team workspace, Task Board and task assignment.

Diagram 3 — message list

#	From	To	Message / action	Type
1. Enter Team Workspace				
1	Member	Flutter App	Select Project from "My Teams"	Call
2	Flutter App	Firestore DB	Fetch Team Info & Members (teams/{projectId})	Call
3	Firestore DB	Flutter App	Return Team Members & Active Roles	Return
4	Flutter App	Member	Display Workspace (Chat, Tasks, Members)	Call
2. Task Management (Task Board)				
5	Member	Flutter App	Navigate to Task Board	Call
6	Flutter App	Firestore DB	Stream Tasks (teams/{projectId}/tasks)	Call
7	Firestore DB	Flutter App	Return Tasks (To Do, In Progress, Done)	Return
3. Create & Assign Task				
8	Member	Flutter App	Tap "Create Task" (Title, Priority, Deadline, Assignee)	Call
9	Flutter App	Firestore DB	Save new task (status: todo)	Call
10	Flutter App	Cloud Functions	Trigger FCM Notification for Assignment	Call
11	Cloud Functions	Firestore DB	Write notification to assigned user (notifications/{assigneeId}/items)	Call
12	Cloud Functions	Assignee	Send Push Notification (New Task Assigned)	Push
4. Update Task Status				
13	Assignee	Flutter App	Drag/Update Task Status (e.g., In Progress -> Done)	Call
14	Flutter App	Firestore DB	Update task status in Firestore	Call
15	Firestore DB	Flutter App	Real-time update reflected on Task Board	Return

7. Diagram 4 — Prompt evaluation and notifications

Two supporting flows: how AI output is rated and how the best prompt version is kept active, and how in-app notifications reach the user and open the right screen.

Participants

Developer · Flutter App · Firestore DB · Cloud Functions

Phases

Phase	Goal	What happens
1. Prompt Evaluation Flow	Collect a rating for every AI output.	The developer receives an AI output (explanation, pitch or setup) and rates it: **Helpful Yes/No**, **Clarity 1–5** and a reason. The app saves it in *feedback/{id}* together with the **promptVersion** and receives a confirmation.
2. Admin / System Prompt Optimization	Keep the best prompt active.	The system analyses feedback grouped by promptVersion: a Cloud Function reads the feedback collection, calculates the **relevance and clarity** metrics and updates the active prompt version in *config/prompts*.
3. Notifications System Flow	Deliver events and deep-link to them.	A new event (request, invite or task update) is written by a Cloud Function. The app streams the unread items from *notifications/{uid}/items*, shows the badge and the Today / Earlier list. Tapping an item parses the payload (**type** and **refId**) through the router and deep-links to the target screen.

ERD mapping

PROMPT_VERSION · PROMPT_EVAL_RUN · AI_GENERATION · AI_FEEDBACK · NOTIFICATION · USER

Business rules shown on the diagram

- Every rating stores the **promptVersion** that produced the output, so versions can be compared fairly.
- The two metrics used for optimisation are **relevance** and **explanation clarity**.
- The **active prompt version** is stored in a configuration document and changed by the system, not by the client.
- Notifications are shown as a **badge plus a Today / Earlier list** of unread items.
- A notification carries **type and refId**, which the router uses to open the target screen directly (deep link).

Diagram 4 — figure

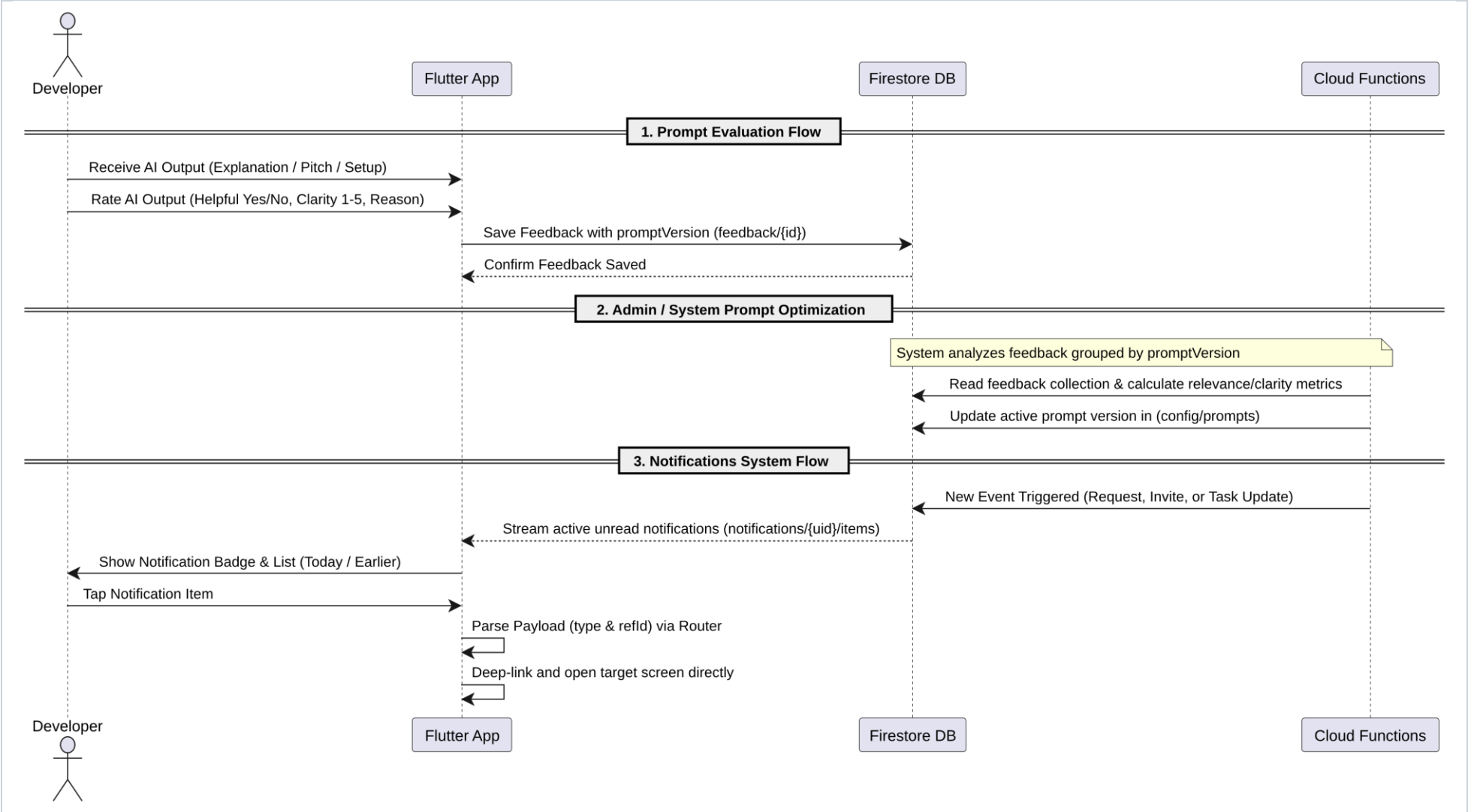


Figure 4. Prompt evaluation, prompt optimisation and the notification flow.

Diagram 4 — message list

#	From	To	Message / action	Type
1. Prompt Evaluation Flow				
1	Developer	Flutter App	Receive AI Output (Explanation / Pitch / Setup)	Call
2	Developer	Flutter App	Rate AI Output (Helpful Yes/No, Clarity 1-5, Reason)	Call
3	Flutter App	Firestore DB	Save Feedback with promptVersion (feedback/{id})	Call
4	Firestore DB	Flutter App	Confirm Feedback Saved	Return
2. Admin / System Prompt Optimization				
Note: System analyzes feedback grouped by promptVersion				
5	Cloud Functions	Firestore DB	Read feedback collection & calculate relevance/clarity metrics	Call
6	Cloud Functions	Firestore DB	Update active prompt version in (config/prompts)	Call
3. Notifications System Flow				
7	Cloud Functions	Firestore DB	New Event Triggered (Request, Invite, or Task Update)	Call
8	Firestore DB	Flutter App	Stream active unread notifications (notifications/{uid}/items)	Return
9	Flutter App	Developer	Show Notification Badge & List (Today / Earlier)	Call
10	Developer	Flutter App	Tap Notification Item	Call
11	Flutter App	Flutter App	Parse Payload (type & refId) via Router	Self
12	Flutter App	Flutter App	Deep-link and open target screen directly	Self

8. Observations and open points

Points where the diagrams and the ERD (DevSync_ERD.pdf) differ or leave something open. They are listed so the team can decide before implementation; none of them changes the flows above.

#	Where	Observation	Suggested decision
1	Diagram 3, step 7	The Task Board shows **To Do, In Progress, Done**, while the ERD and the Core Features define four statuses including **In Review**.	Add the In Review column to the diagram, or remove the status from the ERD.
2	Diagram 2, step 13	Missing roles are checked with **openTrackKeys (filled < count)**; the ERD models this as **slots_needed** versus the members filling each role.	Keep one name for the rule in both documents.
3	Diagram 1, steps 27 and 32–33	Requests are stored under *requests/{projectId}_{uid}*, and the app itself updates the request and adds the member. The ERD uses JOIN_REQUEST with a unique (project, applicant) pair.	Consider doing the accept step (status update + member add + notification) in one Cloud Function so it is atomic.
4	Diagram 1, step 34	The **Acceptance Notification** is drawn as sent by the Flutter App, whereas the other notifications go through a Cloud Function (FCM).	Route it through a Cloud Function as well, for consistency.
5	Diagrams 1–4	Notifications use *notifications/{uid}/items* (a sub-collection), while the ERD has a single NOTIFICATION entity with user_id.	Both are compatible; keep the physical path in the schema file.
6	Diagram 4, phase 2	The phase is named Admin / System Prompt Optimization but only Cloud Functions take part: there is no admin actor and no trigger (schedule or event) shown.	State what starts the analysis (a schedule or a manual run) and where PROMPT_EVAL_RUN results are stored.
7	Diagram 1, participants	**LLM / GitHub API** is drawn as one lifeline although they are two different services.	Split into two lifelines if the diagram is used in the final report.
8	Diagram 2, step 10	The team document is created by the app together with the project save, in two separate writes.	Use a batched write so a project never exists without its team.

How the diagrams cover the 11 core features

Core feature (ERD file)	Covered by
2. Authentication (with Guest Mode) · 3. Profile	Diagram 1, phase 1
4. GitHub Integration	Diagram 1, phase 2
5. AI Match · 6. Project Discovery · 8. Project Details	Diagram 1, phase 3
8. Project Details (Request to Join) · 11. My Projects	Diagram 1, phase 4
9. Create Project · 11. My Projects (invitations)	Diagram 2
10. My Teams	Diagram 3
12. Prompt Evaluation · notifications of features 10 and 11	Diagram 4
1. Splash & Onboarding	No sequence diagram: nothing is stored in the database